

SHRSS AEMaaCS Engineering & Architecture Best Practices Volume (Developers, Architects, Admins)

SHRSS AEMaaCS Engineering & Architecture Best Practices

For developers, technical architects, and AEM administrators

0. How to Use This Volume

This volume is the **engineering lens** on the SHRSS solution:

- It assumes you've read the SHRSS SDD and the all-roles best-practices volume.
- It goes deeper than Experience League on:
 - How SHRSS uses **Content Fragments, GraphQL, Core Components, and tags**.
 - What "good" looks like for **indexes, queries, dispatcher/CDN, and schedulers** in AEMaaCS.
 - How to avoid common **Cloud Service anti-patterns** in a Jobs/Events/News/Locations-heavy solution.

For general AEMaaCS development guidelines, always anchor on:

- [AEM as a Cloud Service Development Guidelines](#)
 - [Implementing Adobe Experience Manager as a Cloud Service](#)
-

1. Platform & Solution Architecture – Mental Model

1.1 SHRSS architecture in one sentence

SHRSS is a **content-driven, CF-centric AEM Sites implementation** on AEMaaCS: pages render local layout and experience; almost all data (jobs, events, news, promotions, locations) is modeled as

Content Fragments, indexed for search, and surfaced via **Core Components + Sling Models**, fronted by **Dispatcher + CDN**.

1.2 Core architectural principles

1. CFs as single source of truth

- Jobs, Events, News, Promotions, Locations, Venue are **Content Fragment models** (see SDD Appendix B).
- Pages never “own” business data; they **reference CFs** or query via GraphQL or search indexes.

2. AEMaaCS-friendly code

- Stateless, cluster-aware, no assumptions about a single node, no local filesystem state.
- All code/config is shipped via Cloud Manager pipelines; no manual changes in /apps or /libs.
See [AEM as a Cloud Service Development Guidelines](#).

3. Separation of concerns

- **Content architects** own CF Models and taxonomies.
- **Developers** own Sling Models, components, queries, indexes.
- **Authors/DAM** own CF and asset instances, tags, and editorial choices.

4. Everything is observable

- Every non-trivial integration and query has:
 - Log lines with correlation IDs.
 - A way to reproduce on development/SDK.
 - A documented failure behavior (what happens if Workday, Unity, etc. fail).
-

2. Codebase Structure & SHRSS Project Conventions

2.1 AEM project modules

SHRSS follows AEM Project Archetype patterns:

- core
Java code: Sling Models, servlets, OSGi services, schedulers, workflow steps.

- `ui.apps`
Component definitions, dialogs, policies, clientlibs, templates.
- `ui.frontend`
Webpack-based front-end build; compiled CSS/JS packaged into clientlibs.
- `ui.content`
Minimal baseline content & configuration; content is **not** treated as mutable here.
- `ui.config`
OSGi configs, repo init, service users/groups/ACLs, runmode-specific configs.
- `dispatcher`
Dispatcher configs (filters, vhosts, cache, rewrites), validated by Cloud Manager.
- `it.tests / ui.tests`
Integration and UI tests wired into Cloud Manager pipelines.

Best practice:

- Keep **business logic** in core (Sling Models/services), not in HTL/JS.
- Keep **environment-specific configuration** in `ui.config`, not core.
- Keep **Dispatcher security rules** in version control, not hand-edited on servers.

For project structure guidance see:

- [AEM Project Structure](#)
-

3. Content Model & Data Contracts

3.1 SHRSS Content Fragment Models as contracts

Each CF Model is a **contract between authors and code**:

- **Events**: eventTitle, eventDescription, eventDate/eventEndDate, eventLocation (CF ref), category, isFeatured, ticketUrl, etc.
- **Jobs**: Workday IDs, title, description, location fields, job URL, tags/category, plus author-controlled fields (image, hot flag, overrides).
- **News**: headline, teaser, body, publish/expiry date, tags, related entities.
- **Locations**: name, address fields, geo data, property type, amenities, hero images.
- **Promotions/Careers**: messaging blocks, CTAs, linked Jobs/Events/Locations.
- **Venue**: facility-level information used by Events, Locations, and site-wide content.

Engineering implications:

- Sling Models should **mirror CF fields 1:1** and expose clear getters.
- Front-end should not guess structure; it should use strongly typed model methods.
- **Breaking changes to CF Models are breaking API changes**; version them or manage migrations.

Relevant content modeling guidance:

- [Learn the Content Modeling Basics for Headless with AEM](#)
- [Defining Content Fragment Models](#)

3.2 SHRSS taxonomy & tags as secondary index

Tags in SHRSS drive:

- Job filters (job category, property, region).
- Event type, target audience, region.
- News categorization (press, corporate, property-specific).
- Location attributes (casino, hotel, dining, entertainment).

Engineering responsibilities:

- Hard-code **tag namespaces and paths** used in components in a single place (constants).
- When tags drive search filters, ensure there is a **matching index rule** for `cq:tags` and relevant CF properties (see §7).
- Avoid depending on **tag titles**; depend on tag IDs (paths).

For tagging framework details:

- [The AEM Tagging Framework](#)
 - [Using Tags](#)
-

4. Component & Template Patterns

4.1 Core component extension strategy

SHRSS uses a **Core Components–first** approach:

- Page, Navigation, Breadcrumb, Language Navigation, Experience Fragment, Button, Teaser, Title, Text, Image, etc.
- Custom SHRSS components:

- Jobs listing + detail.
- Events calendar + detail.
- News listing + article detail.
- Location finder, cards, detail.
- Shared data-driven navigation (e.g., site-wide menus, tiles).

Patterns:

- Extend Core Components via:
 - `sling:resourceSuperType` → preserve upgrades.
 - Sling Models extending core interfaces (e.g., `List`, `Navigation`, `ComponentExporter`).
- Keep **HTL “dumb”**:
 - All logic belongs in Sling Models or services.
 - HTL is limited to branching and simple formatting.

4.2 Template & inheritance strategy

- Editable page templates aligned with:
 - Home, section landing, listing pages, detail pages, utility pages.
- Site root uses a **configuration** that enables:
 - Editable templates.
 - CF Models.
 - Tag namespaces.

Keep template roles clear:

- **Template-only**: structure, allowed components, default policies.
- **Page**: instance-specific composition and configuration.

For template mechanics:

- [Templates to Create Pages that are Editable with the Page Editor](#)
- [The AEM Page Editor](#)

5. Integrations & External Systems

5.1 Workday → Jobs

Pattern:

- Workday exports job data to AEM (via custom integration or middleware).
- Integration writes/updates **Job CFs** under the Jobs folder structure.
- Workday-owned fields are treated as **authoritative**; authors enrich but do not override.

Engineering rules:

- Enforce **read-only** treatment of Workday-controlled fields in the UI (field disabled for authors) where feasible.
- Use **stable identifiers**:
 - Workday Job Requisition ID as the canonical key.
 - Use it for:
 - CF naming.
 - URLs and SEO (when appropriate).
 - Data reconciliation and debugging.
- Integration must be **idempotent**:
 - Re-running the same delta shouldn't create duplicates or diverging data.

5.2 Unity / DPLT / other external APIs

Common patterns:

- Use **service credentials** and OpenAPI-based APIs where available.
See [AEM APIs overview](#).
- Implement external calls in **OSGi services**, not components.
- Put all **API endpoints & credentials** in `ui.config` (OSGi configs, environment variables).

Failure behavior:

- Timeouts and errors **must not break page rendering**:
 - Components should degrade gracefully:
 - Show a fallback message.
 - Hide the widget.
 - Log with a correlation ID.

6. AEMaaS Development Patterns & Anti-Patterns

6.1 Cluster awareness & statelessness

From [AEM as a Cloud Service Development Guidelines](#):

- **Never** store business state in:
 - Java static fields.
 - In-memory caches that aren't repopulated safely.
 - Local filesystem paths.
- Treat every instance as ephemeral:
 - Pods can be killed or rotated anytime.
 - Old and new code can run side-by-side during deployment; code must tolerate both old and new content.

SHRSS examples:

- Job or event caches must be:
 - Rebuildable from CFs or indexes.
 - Scoped per environment, not per node.

6.2 Schedulers & async work

Patterns:

- Use **Sling scheduled jobs**, not custom threading.
- Jobs must be:
 - Idempotent.
 - Safe to run concurrently.
 - Resilient to partial failures (resume on next run).

Anti-patterns:

- "Leader" logic using non-Cloud-safe patterns.
 - Long-running jobs that effectively perform batch exports on every run.
-

7. Indexing, Search & Query Design

Anchor docs:

- [Content Search and Indexing](#)
- [Query and Indexing Best Practices](#)
- [Indexing best practices in AEM](#)

7.1 SHRSS index surface

SHRSS search-driven features include:

- Job listings & filtering.
- Event listings & calendars.
- News search & filtering.
- Location finder & proximity searches (if enabled).
- DAM search for authors.

Design principles:

1. Prefer **few, well-designed indexes** over many bespoke ones:
 - One CF index per major model group (e.g., `shrssJobsLucene`, `shrssEventsLucene`, `shrssNewsLucene`) when necessary.
 - Use property restrictions carefully to avoid "index everything" patterns.
2. Always start from the **query**:
 - Design your query first.
 - Use Explain Query / query debugger on local SDK to see which index is chosen.
 - Add index coverage only where needed.

7.2 What goes in custom indexes

Typical fields for SHRSS CF indexes:

- Common:
 - `jcr:primaryType`, `sling:resourceType`, `cq:tags`
 - contentFragment flags & model references.
- Jobs:
 - Job category, job type, location fields, hot flag, dates, Workday status.
- Events:
 - `eventDate`, `eventEndDate`, category, location reference, `isFeatured`.
- News:
 - `publishDate`, `expiryDate`, categories, tags.

Best practices:

- Mark only searchable/sortable fields as indexed.
- Use `analyzed` for full-text fields, `notAnalyzed` for facets and filters.
- Scope indexes to CF paths (`/content/dam/shrss/content-fragments/...`) to avoid noise.

7.3 Query & component design

For each component that queries:

- **Define max results and paging:**
 - Never return unbounded result sets.
 - For SHRSS lists, 10–20 items per page is the practical ceiling.
- Avoid running queries on every request if:
 - The results can be **pre-computed** (e.g., daily job counts) and stored.
 - You can rely on caching (see §8).

Use the AEMaaCS query guidelines:

- [Query and Indexing Best Practices](#)
-

8. Dispatcher, CDN & Caching Strategy

Core reference:

- [AEM as a Cloud Service caching](#)

8.1 Multi-layer cache view

For SHRSS:

1. **Adobe-managed CDN** – primary cache for public pages and assets.
2. **Dispatcher (Apache + dispatcher module)** – path and header-driven cache rules.
3. **Publish instances** – minimal work; serve only cache misses.

Goals:

- Maximize cacheability of:
 - Listing pages (jobs/events/news).
 - Category/landing pages.
 - Location details and “evergreen” content.
- Minimize dynamic page generation:
 - Use persisted queries or structured URLs instead of arbitrary query strings where possible.

8.2 Dispatcher configuration patterns

Key rules for SHRSS:

- Block access to:
 - `/system/console`, `/crx/*`, `/libs/*`, `/apps/*`, `/oak:index/*`, etc.
- Only allow:
 - Public endpoints (`/content/shrss/...`, `/etc.clientlibs/...`, `assets`).
 - Specific servlets/APIs (e.g., GraphQL, headless APIs) using **whitelists**.

Anti-patterns:

- Overly generic allow rules (`/content/*`) without matching deny rules.
 - Allowing POST/PUT/DELETE to publish via dispatcher.
 - Serving JSON APIs without proper cache headers (either cache them intentionally or mark no-store).
-

9. Testing & Quality Gates

9.1 Test pyramid

For SHRSS:

1. **Unit tests** (core):
 - Business logic, Sling Models, helpers.
2. **Integration tests** (`it.tests`):
 - HTTP calls against embedded AEM, verifying component rendering and APIs.
3. **UI tests** (`ui.tests` – e.g., Cypress):
 - Cloud Manager custom UI tests.
4. **Performance & indexing checks**:
 - At least one synthetic scenario for critical queries (job search, event list).

Align with Cloud Manager guidance:

- [Query and Indexing Best Practices](#)
-

10. Troubleshooting & Operations

10.1 Everyday tools

- **Developer Console:**
 - Health checks, Sling logs, status dumps, OSGi bundles.
- **Cloud Manager logs:**
 - Build logs, environment logs, dispatcher logs.
- **Local SDK:**
 - Reproduce issues, inspect indexes and queries, run Explain Query.

Anchor doc:

- [Operations and Maintenance on Experience Manager as a Cloud Service](#)

10.2 Common SHRSS scenarios

- **“Jobs disappeared from listing”:**
 - Check:
 - CFs still published and active (Workday status).
 - Tags/categories used by the list configuration.
 - Index reindex status (look for query warnings and index logs).
- **Slow event listing:**
 - Run Explain Query on SDK.
 - Confirm correct index coverage and that you’re not accidentally traversing CF references without constraints.

References

- [AEM as a Cloud Service Development Guidelines](#)
- [Implementing Adobe Experience Manager as a Cloud Service](#)
- [Content Search and Indexing](#)
- [Query and Indexing Best Practices](#)
- [Indexing best practices in AEM](#)
- [AEM as a Cloud Service caching](#)
- [AEM APIs overview](#)
- [The AEM Tagging Framework](#)
- [Using Tags](#)
- [Defining Content Fragment Models](#)